10. Bölüm

Yapılar, Birlikler ve İsim Uzayları

10.1 Yapılar

Yapısal programlamaya adını veren yapı (structure), türetilmiş (derived) veya kullanıcı tanımlı (user defined) bir veri tipidir. Farklı tiplerdeki elemanları bir arada gruplandıran özel bir veri tipi tanımlamak için `struct` anahtar kelimesini kullanırız. Yapı bir veya birden fazla ilkel veri tipinin (`char`, `int`, `long`, `float`, `double`, ...) ve bu tiplere ait dizilerin bir araya gelmesiyle oluşturulan yeni veri tipleridir.

Diziler, aynı tipte elemanlardan oluşmasına rağmen, yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Aşağıda sözde kodu verilmiştir;

```
struct yapı-kimliği {
 veri-tipi yapı-elemanı-kimliği1;
 veri-tipi yapı-elemanı-kimliği1;
 ...
};
```

Örnek olarak `adı`, `soyadı`, `numarası`, `yaşı`, `cinsiyeti` gibi farklı öğeler ile bir `Ogrenci` yapısı tanımlanabilir;

```
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1: Erkek, 2: Kadın
};
```

Yapı tanımlandıktan sonra aşağıdaki sözde kodda verildiği gibi yeni değişkenler tanımlanabilir;

```
yapı-kimliği değişken-kimliği[,değişken-kimliği2][,değişken-kimliği3];
```

Yapının elemanlarına tanımlama sırasında **başlangıç değeri** (initial value) verilebilir yani yapı değişkeni bazı değerlerle **başlatılabilir** (initialize). Bunun için diziler gibi süslü parantez kullanılır;

```
Ogrenci ogrenci1; // Başlatılmamış
Ogrenci ogrenci2={"Deniz", "AK", 35, 2};
Ogrenci ogrenci3={"Damla", "YILDIZ", 28, 2};
```

Bir yapının içinde bir başka yapı da tanımlanabilir. Yani iç içe yapılar (nested structs) tanımlanabilir.

10.2 Yapı Elemanlarına Erişim

Tanımlanan yapı değişkenleri üzerinden her bir alamana **nokta işleci** (dot operator) erişilir. Bu işlecin önceliği parantez ile aynıdır. Atama işleci (`=`) bir yapıyı doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz.

```
#include <iostream>
#include <string> // string sınıfı için gerekli
struct Ogrenci { // C++'ta tür isimlerini büyük harfle başlatmak yaygındır
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1: Erkek, 2: Kadın
};
int main() {
    // ogrenci1 yapısı başlatılmadığından değerleri aşağıdaki gibi güncelleniyor:
    // C++ std::string ile doğrudan atama yapabiliriz, strcpy'ye gerek yok.
    Ogrenci ogrenci1;
    ogrenci1.adi = "Ilhan";
    ogrenci1.soyadi = "OZKAN";
```

```
    ogrenci1.yas = 50;
    ogrenci1.cinsiyet = 1;
    std::cout << "Ogrenci 1 Bilgileri:" << std::endl;
    std::cout << "Adi: " << ogrenci1.adi << std::endl;  // nokta işleci ile adi alanına erişildi.
    std::cout << "Soyadi: " << ogrenci1.soyadi << std::endl; // nokta işleci ile soyadi alanına
     ↪ erişildi.
    std::cout << "Yasi: " << ogrenci1.yas << std::endl; // nokta işleci ile yas alanına erişildi.
    std::cout << "Cinsiyeti: " << ogrenci1.cinsiyet << std::endl; // nokta işleci ile cinsiyet
     ↪ alanına erişildi.
    std::cout << "-------------------" << std::endl;
    // yapı bazı değerlerle başlatıldı. 'struct' anahtar kelimesi C++'ta zorunlu değil.
    Ogrenci ogrenci2 = {"Deniz", "AK", 35, 2};
    std::cout << "Ogrenci 2 Bilgileri:" << std::endl;
    std::cout << "Adi: " << ogrenci2.adi << std::endl;
    std::cout << "Soyadi: " << ogrenci2.soyadi << std::endl;
    std::cout << "Yasi: " << ogrenci2.yas << std::endl;
    std::cout << "Cinsiyeti: " << ogrenci2.cinsiyet << std::endl;
    std::cout << "-------------------" << std::endl;
    // yapı kopyalama (sığ kopyalama)
    Ogrenci ogrenci3 = ogrenci2;
    std::cout << "Ogrenci 3 Bilgileri (Ogrenci 2'den kopyalandi):" << std::endl;
    std::cout << "Adi: " << ogrenci3.adi << std::endl;
    std::cout << "Soyadi: " << ogrenci3.soyadi << std::endl;
    std::cout << "Yasi: " << ogrenci3.yas << std::endl;
    std::cout << "Cinsiyeti: " << ogrenci3.cinsiyet << std::endl;
}
```

10.3 Yapı Göstericileri

Yapı göstericileri, tıpkı diğer değişkenlere gösterici tanımladığımız gibi tanımlayabiliriz. Gösterici üzerinden yapı değişkenlerine **dolaylı işleç** (indirection operator) yani (->) ile erişilir. Bu nokta işleci ile aynı önceliklidir.

Yapılar ve göstericileri; veri tabanları, dosya yönetim uygulamaları ve ağaç ve bağlı listeler gibi karmaşık veri yapılarını işlemek gibi farklı uygulamalarda kullanılır.

```
#include <iostream> // std::cout için
#include <string>   // std::string için
// C++'ta struct isimlerini Büyük Harfle başlatmak yaygındır
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1: Erkek, 2: Kadın
};
int main() {
    Ogrenci ogrenci1;
    // Alanlar std::string tanımlandığından strcpy() yerine doğrudan atama yapabiliriz. Çok daha
     ↪ güvenli.
    ogrenci1.adi = "Ilhan";
    ogrenci1.soyadi = "OZKAN";
    ogrenci1.yas = 50;
    ogrenci1.cinsiyet = 1;
    // İşaretçi tanımı. Yine 'struct' kelimesi yok.
    Ogrenci* ogrenciGosterici;
    ogrenciGosterici = &ogrenci1;
    // nokta işleci yerine '->' işleci kullanılır.
    std::cout << "Adi: " << ogrenciGosterici->adi << std::endl
    << "Soyad: " << ogrenciGosterici->soyadi << std::endl
    << "Yasi: " << ogrenciGosterici->yas << std::endl
    << "Cinsiyeti: " << ogrenciGosterici->cinsiyet << std::endl;
}
```

10.4 Yapılar ve Fonksiyonlar

Sıradan dizilere benzer şekilde bir yapı dizisi tanımlayabiliriz. Ayrıca yapı değişkenini bir fonksiyona parametre olarak gönderebilir ve bir fonksiyondan bir yapı döndürebilirsiniz;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // int yerine ileride göreceğimiz doğrudan enum tipi kullanmak daha güvenli
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
    Ogrenci ogrenciler[30]; // 30 elemanlı Öğrenci dizisi
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    // std::string ile doğrudan atama yapabiliriz. ERKEK enum değeri tam sayı gibi davranabilir.
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 2};
    return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
    std::cout << "Ogrenci Bilgileri:" << std::endl;
    std::cout << "Adi: " << pOgrenci.adi << std::endl;
    std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
    std::cout << "Yasi: " << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << " (0:Belirtilmemis, 1:Kadin, 2:Erkek)" <<
    ↪ std::endl;
    std::cout << "-------------------" << std::endl;
}
```

Yapı elemanları değer tipler olduğundan, yapılara ait referansları parametre olarak kullanmak, olabilecek değişiklikleri aktarmak için üstünlük sağlar. Gösterici kullanımı tehlikelidir;

```
#include <iostream>
#include <string> // std::string sınıfı için gerekli
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet;
};
Ogrenci yeniOgrenci();
void ogrenciOku(Ogrenci& pOgrenci);
void ogrenciYaz(const Ogrenci& pOgrenci);
int main() {
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
    ogrenciOku(o);
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1};
    return yeni;
}
void ogrenciOku(Ogrenci& pOgrenci) {
    std::cout << "Adi Giriniz: ";
    std::cin >> pOgrenci.adi; //referans ile de üyerlere nokta işleci ile erişilir.
    std::cout << "Soyadi Giriniz: ";
    std::cin >> pOgrenci.soyadi;
```

```
    std::cout << "Yas Giriniz: ";
    std::cin >> pOgrenci.yas;
    std::cout << "Cinsiyet Giriniz (0:Belirtilmemis, 1:Erkek, 2:Kadin): ";
    std::cin >> pOgrenci.cinsiyet;
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrenciYaz(const Ogrenci& pOgrenci) {
    std::cout << "-------------------" << std::endl;
    std::cout << "Ogrenci Bilgileri:" << std::endl;
    std::cout << "Adi: " << pOgrenci.adi << std::endl;
    std::cout << "Soyadi: " << pOgrenci.soyadi << std::endl;
    std::cout << "Yasi: " << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyeti: " << pOgrenci.cinsiyet << std::endl;
    std::cout << "-------------------" << std::endl;
}
```

10.5 İç İçe Yapılar

İç içe yapılara (nested structs) ait `Ogrenci` yapımıza doğum tarihini içerecek yapı eklenen örnek, aşağıda verilmiştir;

```
#include <iostream>
#include <string> // std::string için gerekli
// C++'ta yapı isimlerini Büyük Harfle başlatmak yaygındır (Ogrenci)
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    int cinsiyet; // 1:erkek, 2:Kadın, 0:Belirtmiyor
    // İç yapıyı isimlendirmek ve bir üye olarak tanımlamak modern C++'ta daha temiz ve
     ↪ güvenlidir.
    struct Tarih {
        int gun;
        int ay;
        int yil;
    } dogumTarihi;
};
Ogrenci yeniOgrenci();
void ogrenciYaz(Ogrenci);
int main() {
    Ogrenci o = yeniOgrenci();
    ogrenciYaz(o);
}
Ogrenci yeniOgrenci() {
    Ogrenci yeni = {"Ilhan", "Ozkan", 50, 1, {1, 1, 1970}};
    return yeni;
}
void ogrenciYaz(Ogrenci pOgrenci) {
    std::cout << pOgrenci.adi << std::endl;
    std::cout << pOgrenci.soyadi << std::endl;
    std::cout << "Yas:" << pOgrenci.yas << std::endl;
    std::cout << "Cinsiyet:" << pOgrenci.cinsiyet << std::endl;
    // İsimlendirilmiş iç yapı kullandığımız için erişim 'dogumTarihi' üyesi üzerinden yapılır.
    std::cout << "Dogum Tarihi:" << pOgrenci.dogumTarihi.gun << "-" << pOgrenci.dogumTarihi.ay <<
     ↪ "-" << pOgrenci.dogumTarihi.yil << std::endl;
}
```

10.6 Öz Referanslı Yapılar

Öz referanslı yapı (self-referetial struct), öğelerinden bir veya daha fazlası kendi tipindeki göstericilerden oluşan bir yapıdır. Öz referanslı yapılar, bağlantılı listeler ve ağaçlar gibi karmaşık olan **dinamik veri yapıları** (dynamic data structure) içinde yaygın olarak kullanılırlar.

```
#include <iostream> // std::cout için
```

```
#include <string>   // std::string için
struct Ogrenci {
    std::string adi;
    std::string soyadi;
    int yas;
    Ogrenci* sonrakiOgrenci; // Kendi veri tipinden bir yapı göstericisi
};
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci*);
int main() {
    // std::string ve nullptr kullanımı ile temiz bir başlatma
    // İlk düğüm: Ilhan
    Ogrenci ilk = {"Ilhan", "OZKAN", 50, nullptr};
    // İkinci düğüm: Ayse
    Ogrenci sonraki = {"Ayse", "YILMAZ", 40, nullptr};
    // Bağlantı: Ilhan'dan Ayse'ye
    ilk.sonrakiOgrenci = &sonraki;
    // Üçüncü düğüm: Tahsin
    Ogrenci birsonraki = {"Tahsin", "BULUT", 35, nullptr};
    // Bağlantı: Ayse'den Tahsin'e
    sonraki.sonrakiOgrenci = &birsonraki;
    // Listenin başını yazdır fonksiyonuna gönderiyoruz
    ogrencileriYaz(&ilk);
}
// 'const Ogrenci*' yapının içeriğinin değiştirilmesini engeller (güvenli).
void ogrencileriYaz(const Ogrenci* pIlk) {
    int sayac = 0;
    while (pIlk != nullptr) {
        std::cout << "OGRENCI " << ++sayac << ": "
        << "Adi: " << pIlk->adi << " "
        << "Soyad: " << pIlk->soyadi << " "
        << "Yas: " << pIlk->yas << std::endl;

        // Bir sonraki düğüme geçiş
        pIlk = pIlk->sonrakiOgrenci;
    }
}
```

10.7 Yapı Dolgusu

C++ dilinde **yapı dolgusu** (structure padding), işlemci (CPU) mimarisi ile belirlenir. Dolgu işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. Bunun nedeni 16, 32 veya 64 bitlik bir bilgisayarda işlemcinin tek seferde bellekten okuyacağı bayt sayısının farklı okumasından kaynaklanmaktadır.

```
#include <iostream>
struct Yapi1 {
    char a;
    char b;
    int c;
};
struct Yapi2 {
    char d;
    int e;
    char f;
};
int main() {
    std::cout << "char bellek miktari: " << sizeof(char) << std::endl;
    // char bellek miktarı: 1
    std::cout << "int bellek miktari: " << sizeof(int) << std::endl;
    // int bellek miktarı: 4
    std::cout << "------------------------" << std::endl;
    // Teorik beklenen boyut
```

```
    std::cout << "Yapilardaki 2 adet char ve 1 adet int olmasi gereken bellek miktari : "
    << (2 * sizeof(char) + sizeof(int)) << std::endl;
    // yapilar için olmasi gereken bellek miktarı: 6
    // Gerçek bellek boyutları (bellek hizalaması nedeniyle farklılık gösterir)
    std::cout << "Yapi1 icin bellekte ayrilan miktar: " << sizeof(Yapi1) << std::endl;
    // Yapi1 için bellekte ayrılan miktar: 8 (1bayt a + 1bayt b + 2bayt dolgu + 4bayt c)
    std::cout << "Yapi2 icin bellekte ayrilan miktar: " << sizeof(Yapi2) << std::endl;
    // Yapi2 için bellekte ayrılan miktar: 12 (1bayt d + 3bayt dolgu + 4bayt e + 1bayt f + 3bayt
     ↪ dolgu)
}
```

Yukarıda verilen örnekte aynı bellek miktarına sahip elemanlar için yapının bütününe bakıldığında farklı miktarda bellek ayrıldığı anlaşılmaktadır. Dolgu (padding) işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. **Kod içerisindeki tanımlanacak tüm yapılar için bunu engellemenin yolu** aşağıdaki ön işlemci yönergesini (preprocessor directive) kaynak koda eklemektir;

```
#pragma pack(1)
```

Eğer yalnızca belirlenen bir yapı için bunun yapılması isteniyorsa yapı tanımına `__attribute__((packed))` özelliği eklenir. Bu komutlar C dilinde olduğu gibi C++'ta da aynı şekilde çalışır, ancak bunlar standart C++'ın bir parçası değildir ve derleyiciden derleyiciye (GCC, Clang, MSVC) farklılık gösterebilir. Aşağıdaki kod GCC ve Clang tabanlı derleyicilerde sorunsuz çalışacaktır. Yapı dolgusuna ilişkin program örneği aşağıda verilmiştir;

```
#include <iostream>
#pragma pack(1) // Tüm yapılarda dolgu baytlarını devre dışı bırakır (Derleyiciye özgü)
struct Yapi1 {
    char a;
    char b;
    int c;
};
struct __attribute__((packed)) Yapi2 {
    char a;
    int b;
    char c;
};
int main() {
    std::cout << "char bellek miktari: " << sizeof(char) << std::endl;
    // char bellek miktarı: 1
    std::cout << "int bellek miktari: " << sizeof(int) << std::endl;
    // int bellek miktarı: 4
    std::cout << "------------------------" << std::endl;
    // Teorik beklenen boyut (paketlenmiş yapılar için)
    std::cout << "Yapilardaki 2 adet char ve 1 adet int olmasi gereken bellek miktari: "
    << (2 * sizeof(char) + sizeof(int)) << std::endl;
    // yapılar için olması gereken bellek miktarı: 6
    // Gerçek bellek boyutları (paketlenmiş oldukları için 6 olmalıdır)
    std::cout << "Yapi1 icin bellekte ayrilan miktar: " << sizeof(Yapi1) << std::endl;
    // Yapi1 için bellekte ayrılan miktar: 6 (paketlenmiş)
    std::cout << "Yapi2 icin bellekte ayrilan miktar: " << sizeof(Yapi2) << std::endl;
    // Yapi2 için bellekte ayrılan miktar: 6 (paketlenmiş)
}
```

10.8 Yapılarda Bit Alanları

Bit alanları (bit field), boyutu azaltmak için C ve C++ dili yapılarını (struct) sıkıca paketler. Yapı üyeleri için bit sayısı verilir ve derleyici bit düzeyinde alanları uyarlar. Bu nedenle bit alan üyesinin adresini alınamaz! `sizeof()` kullanımına da izin verilmez!

```
struct Date
{
    unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
    unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
    unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};
```

Tüm yapı sadece 22 bit kullanıyor ve normal derleyici ayarlarıyla bu yapının boyutu 4 bayt olur. Kullanımı oldukça basittir. Sadece değişkeni bildirimi yapılır ve sıradan bir yapı gibi kullanılır;

```
#include <iostream>
struct Date
{
    unsigned int Year : 13; // 2^13 = 8192, Yıl değerini tutmak için yeterlidir
    unsigned int Month: 4;  // 2^4 = 16, Ay değerini tutmak için yeterlidir.
    unsigned int Day:   5;  // 2^5 = 32, Gün değerini tutmak için yeterlidir
};
int main() {
    Date d;
    d.Year = 2025;
    d.Month = 2;
    d.Day =  28;
    std::cout << "Year:" << d.Year << std::endl //2025
    << "Month:" << d.Month << std::endl //2
    <<  "Day:" << d.Day << std::endl; //28
    std::cout << sizeof d << std::endl; // 4
    //std::cout << sizeof d.Year << std::endl; HATA Olur!
}
```

10.9 Birlikler

Birlikler (union), yapılar (struct) gibi tanımlanır. Aralarındaki fark **yapı elemanlarının her birine ayrı bellek bölgesi ayrılırken, birlik üyelerinin her biri aynı bellek bölgesini paylaşırlar.** Birlik tanımına ilişkin sözde kod aşağıda verilmiştir;

```
union yapı-kimliği {
 veri-tipi yapı-elemanı-kimliği1;
 veri-tipi yapı-elemanı-kimliği1;
 ...
};
```

Birliğin bellek boyutu, elemanlarından en fazla bellek kaplayanı kadardır.

Aşağıda üyelerinin aynı bellek bölgesini paylaştığını gösteren program örneği bulunmaktadır;

```
#include <iostream>
#include <iomanip>
union Birlik { // C++'ta union isimlerini Büyük Harfle başlatmak yaygındır
    char baytlar[4];
    int tamsayi;
    char bayt;
    short int kisatamsayi;
};
int main() {
    Birlik b;
    // Boyut kontrolleri: birlik üyeleri kaç bayt?
    std::cout << "sizeof Birlik.baytlar: " << sizeof(b.baytlar) << std::endl;
    std::cout << "sizeof Birlik.bayt: " << sizeof(b.bayt) << std::endl;
    std::cout << "sizeof Birlik.tamsayi: " << sizeof(b.tamsayi) << std::endl;
    std::cout << "sizeof Birlik.kisatamsayi: " << sizeof(b.kisatamsayi) << std::endl;
    std::cout << "sizeof Birlik: " << sizeof(Birlik) << std::endl;
    b.tamsayi = 0x1B2C3D4F; // b'in tam sayı üyesi değişiyor. Bakalım diğer üyeler ne oldu?
    std::cout << "------------------------" << std::endl;
    std::cout << "b.tamsayi: 0x" << std::hex << std::setw(8) << std::setfill('0') << b.tamsayi <<
     ↪  std::dec << std::endl;
    // char türlerini hex olarak yazdırmak için (int) cast'i gereklidir, aksi takdirde karakter
     ↪  olarak yazdırılırlar.
```

```
    std::cout << "b.baytlar: " << std::hex << std::setw(2) << std::setfill('0') << (int)(unsigned
     ↪ char)b.baytlar[0] << "-" << std::hex << std::setw(2) << std::setfill('0') <<
     ↪ (int)(unsigned char)b.baytlar[1] << "-" << std::hex << std::setw(2) << std::setfill('0')
     ↪ << (int)(unsigned char)b.baytlar[2] << "-" << std::hex << std::setw(2) <<
     ↪ std::setfill('0') << (int)(unsigned char)b.baytlar[3] << std::dec << std::endl;
    std::cout << "b.bayt: 0x" << std::hex << std::setw(2) << std::setfill('0') << (int)(unsigned
     ↪ char)b.bayt << std::dec << std::endl;
    std::cout << "b.kisatamsayi: 0x" << std::hex << std::setw(4) << std::setfill('0') <<
     ↪ b.kisatamsayi << std::dec << std::endl;
    std::cout << "------------------------" << std::endl;
    // Üyelerden biri değiştirildiğinde diğer tüm üyelerin içeriği değişir!
    b.bayt = 0x11;
    std::cout << "Değiştikten Sonra: b.tamsayi: 0x" << std::hex << std::setw(8) <<
     ↪ std::setfill('0') << b.tamsayi << std::dec << std::endl;
}
/*Program Çıktısı:
sizeof Birlik.baytlar: 4
sizeof Birlik.bayt: 1
sizeof Birlik.tamsayi: 4
sizeof Birlik.kisatamsayi: 2
sizeof Birlik: 4
------------------------
b.tamsayi: 0x1b2c3d4f
b.baytlar: 4f-3d-2c-1b
b.bayt: 0x4f
b.kisatamsayi: 0x3d4f
------------------------
Değiştikten Sonra: b.tamsayi: 0x1b2c3d11
*/
```

10.10 İsim Uzayları

İsim uzayları (namespace), değişkenleri (variable), sabitler (constexpr), fonksiyonları (function), yapıları (struct), birlikleri (union) ve ileride göreceğimiz sınıfları (class) bir isim altında toplayabileceğimiz bir tanımlamadır. Birden fazla kütüphane kullanırken isim çakışmalarını önlemek için kullanılır. Böylece aynı kimlikli sınıf, değişken ve fonksiyon birden fazla isim uzayında bulunabilir.

Buna bir örnek verelim. `fonk()` kimlikli bir fonksiyonu olan bir kod yazıyor olabilirsiniz ve aynı fonksiyon aynı kimlikle başka bir başlıkta (header) da mevcut olabilir. Bu durumda derleyicinin, kodunuz içinde hangi `fonk()` fonksiyona atıfta bulunduğunuzu bilmesinin bir yolu yoktur. İsim uzayları, bu zorluğun üstesinden gelmek için tasarlanmıştır ve farklı kütüphanelerde bulunan aynı kimliğe sahip benzer fonksiyonları, sınıfları, değişkenleri vb. ayırt etmek için ek bilgi olarak kullanılır. İsim uzaylarına en iyi örnek, C++ standart kütüphanesidir (`std`). Bu nedenle, C++ programı yazarken her seferinde `std::` yazmamak için kodun başında `using namespace std;` yönergesini ekleriz;

§10.10.1 İsim Uzayı Tanımlama

Bir isim uzayının tanımı, aşağıdaki gibi `namespace` anahtar sözcüğüyle tanımlanır;

```
namespace  isim-uzayı-kimliği
{
 // kod;
 // -değişken tanımlama (int a,b;)
 // -fonksiyon tanımlama (void topla();)
 // -yapı tanımlama (struct Yapi{ /* ... ?/ };)
 // -birlik tanımlama (union Yapi{ /* ... ?/ };)
 // -sınıf tanımlama (class Kisi{ /* ... ?/ };)
}
```

Burada isim uzayına ilişkin bloğun noktalı virgül ile bitmediği gözden kaçırılmamalıdır. Çünkü **isim uzayına ait bloğun görevi tanımlamaları gruplamaktır.** İsim uzayının içindeki bir öğeye erişmek için **kapsam çözümleme işleci** (scope resolution operator) kullanılır. Ayrıca `using namespace` yönergesini (directive) kullanarak isim uzaylarını ile kapsam çözümleme işlecinin önek olarak eklenmesini de önleyebilirsiniz.

```
#include <iostream>
```

```
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
 ↪ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */
namespace birinci // birinci kimlikli isim uzayı
{
    int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "Birinci isim uzayındaki fonksiyon." << endl;
    }
}
namespace ikinci // ikinci kimlikli isim uzayı
{
    int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "İkinci isim uzayındaki fonksiyon." << endl;
    }
}
using namespace birinci;  /* birinci:: isim uzayını kullanmak için yönerge: Bu yönerge ile
 ↪ "birinci::tamsayi" gibi talimatlar sadece "tamsayi" şeklinde kullanılabilir. */
int main () {
    fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
    cout << tamsayi << endl; // birinci isim uzayındaki tamsayi kullanılır.
    ikinci::fonksiyon(); // kapsam çözümleme işleci ile ikinci isim uzayındaki fonksiyon
     ↪ çağrılır.
    cout << ikinci::tamsayi << endl; // kapsam çözümleme işleci ile ikinci isim uzayındaki
     ↪ tamsayi kullanılır.
}
/* Programın Çıktısı:
Birinci isim uzayındaki fonksiyon.
100
İkinci isim uzayındaki fonksiyon.
200
*/
```

İsim uzayları, aşağıdaki şekilde iç içe (nested) de yerleştirilebilir;

```
#include <iostream>
using namespace std; /* std:: isim uzayını kullanmak için yönerge: Bu yönerge ile "std::cout"
 ↪ gibi talimatlar sadece "cout" şeklinde kullanılabilir. */

namespace birinci // birinci kimlikli isim uzayı
{
    int tamsayi=100; // Birinci isim uzayındaki tamsayi değişkeni
    void fonksiyon() {
        cout << "Birinci isim uzayındaki fonksiyon." << endl;
    }
    namespace ikinci // ikinci kimlikli iç isim uzayı
    {
        int tamsayi=200; // ikinci isim uzayındaki tamsayi değişkeni
        void fonksiyon() {
            cout << "İkinci isim uzayındaki fonksiyon." << endl;
        }
    }
}
using namespace birinci::ikinci; /* birinci::ikinci:: iç isim uzayını kullanmak için yönerge */
int main () {
    birinci::fonksiyon(); // birinci isim uzayındaki fonksiyon çağrılır.
    cout << birinci::tamsayi << endl;
    fonksiyon(); // birinci isim uzayı içindeki ikinci isim uzayındaki fonksiyon çağrılır.
    cout << tamsayi << endl;
}
/* Program çalıştığında:
Birinci isim uzayındaki fonksiyon.
100
```

```
İkinci isim uzayındaki fonksiyon.
200
*/
```

§10.10.2 Bağımlı Argüman Arama

Açık bir isim uzayı yönergesi olmadan bir fonksiyonu çağırırken, derleyici o işlevin prototipinden biri de o isim uzayında ise, o isim uzayı içindeki bir işlevi çağırmayı seçebilir. Buna **bağımlı argüman arama** ADL (argument dependent lookup) denir;

```
namespace Test
{
    int func(int i);
    struct Yapi {
        //...
    };
    int func2(const Yapi &data);
}
int main() {
    //...
    func(5); /*Hata alınır. Hangi İsim Uzayından olduğu belli değil. Çünkü "int func(int);"
     ↪ şeklide prototipi olan bir sürü fonksiyon vardır. */

    Test::Yapi data;
    func2(data); /* Hata alınmaz. Kullandığı argümana göre isim uzayı belli. Çünkü "int
     ↪ func(const Yapi &);" şeklide prototipi olan tek bir fonksiyon vardır. */
}
```

> En uygun kullanım, isim uzayı adı ile kapsam çözümleme işlecini birlikte kullanmaktır.

§10.10.3 İsim Uzaylarını Genişletme

İsim alanlarının kullanışlı bir özelliği de onları genişletebilmemizdir. Daha sonradan başka bir dosyada bile isim uzayına yeni üyeler ekleyebilirsiniz. Buna örnek olarak `<iostream>` ve `<string>` başlık dosyaları verilebilir. Birinci kaynak dosyamız aşağıdaki gibi olabilir;

```
//TEST1.CPP Dosyası:
namespace Test
{
    void fonk1() {}
}
//arada başka kodlar yer alabilir
namespace Test
{
    void fonk2() {}
}
```

İkinci dosyamız da aşağıdaki gibi olabilir;

```
//TEST2.CPP Dosyası:
    namespace Test
    {
        void fonk3() {}
    }
    //arada başka kodlar yer alabilir
    namespace Test
    {
        void fonk4() {}
    }
```

§10.10.4 using Anahtar Kelimesinin İsim Uzaylarında Kullanılması

Bir `using` yönergesi, `namespace` anahtar sözcüğüyle birleştirildiğinde o isim uzayını belirtmeden içindeki kullanırsınız.

```
namespace Test
```

```
{
    void func() {}
    void func2() {}
}
int main( {
    //...
    Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
    //...
    using namespace Test; /* Test isim uzayını bu yönerge ile projemize dahil ediyoruz. Artık
     ↪ kapsam çözümleme işlecini kullanmamıza gerek kalmaz. */
    func();
    func2();
    //...
}
```

Tüm isim uzayı yerine, isim uzayındaki seçili üyeleri projeye dahil etmek de mümkündür;

```
namespace Test
{
    void func() {}
    void func2() {}
}
int main( {
    //...
    Test::func2(); //kapsam çözümleme işleci kullanarak istediğimiz üyeyi kullanabiliriz
    //...
    using namespace Test::func; /* Test isim uzayının içindeki func fonksiyonunu bu yönerge ile
     ↪ projemize dahil ediyoruz. Artık bu fonksiyon için kapsam çözümleme işlecini kullanmamıza
     ↪ gerek kalmaz. */
    func(); //Başarılı bir şekilde projeye dahil edilmiştir.
    func2(); //Hata: Bu fonksiyon projeye dahil edilmemiştir.
    //...
}
```

Özellikle başlık (header) dosyalarında `using namespace` kullanmak çoğu durumda kötü görülür. Bu yapılırsa, isim uzayı dahil edilmiş başlığı içeren her dosya projeye aktarılır. Bu da çatışmalara yol açabilir. Birinci başlık dosyamız söyle olsun;

```
\\BIRINCI.H Başllık Dosyası:
namespace AAA
{
    class C;
}
```

İkinci başlık dosyamız da söyle olsun;

```
\\IKINCI.H Başllık Dosyası:
namespace BBB
{
    class C;
}
```

Birinci başlşık dosyasını kullanan bir başka başlık dosyamız aşağıdaki gibi olsun;

```
//UCUNCU.h Baslik Dosyasi:
using namespace AAA;
```

Son olarak bu başlık dosyalarını kullanan ana dosyamız şöyle olsun;

```
//MAIN.CPP Dosyasi:
#include "IKINCI.h"
#include "UCUNCU.h"
using namespace AAA;
int main() {
    //...
    C c; // Hata: BBB::C mi? AAA::C mi?.
    //...
}
```

10.11 Düşün ve Çöz

- **Düşün**: Yapı Dolgusu kavramını açıklayınız. İşlemcinin veriye erişim hızı ile bellek tasarrufu arasındaki denge bu noktada nasıl kurulur?
- **Düşün**: Birlik (union), neden aynı anda sadece bir üyenin değerini tutabilir? Bellek paylaşımı mantığını bir şema ile açıklayınız.
- **Düşün**: Bit alanları (bit field) kullanımı, düşük seviyeli donanım kontrolünde (register) neden vazgeçilmezdir?
- **Çöz**: Bir öğrencinin adını, numarasını ve üç dersinin notunu tutan bir `struct` tanımlayınız. Bu yapıdan bir dizi oluşturup sınıf ortalamasını hesaplayınız.
- **Çöz**: İç içe yapılar (nested structs) kullanarak bir "Şirket" yapısı ve bu yapının içinde "Departman" ve "Çalışan" bilgilerini tutan bir model oluşturunuz.
- **Çöz**: Bir `union` kullanarak, 4 baytlık bir `int` sayının bellekteki her bir baytına karakter göstericisiyle erişen bir program yazınız.
- **Çöz** İsim çakışması (name collision) sorunu büyük projelerde neden kritik bir probleme dönüşür? İsim uzaylarının (namespace) bu sorunu nasıl çözdüğünü somut bir örnekle gösterin.
- **Düşün**: `using namespace std;` direktifinin kullanımının neden büyük projelerde sorunlara yol açabileceğini açıklayın. Alternatif olarak ne kullanılması önerilir?
- **Düşün**: Bağımlı argüman arama ADL (argument dependent lookup) nedir? `operator<<` gibi işleçlerin neden doğru ad uzayında bulunabildiğini ADL mekanizmasıyla açıklayın.